# AICTE FDR portal implementation blueprint

Oct 24, 2026 · @kaka

## 1. Problem statement

SIH 26265 asks for one secure portal that tracks every security deposit and FDR from submission to release, and that keeps what an institution claims separate from what a bank confirms. The official text is narrower and more cautious than the framing the three research reports used, so the build should follow the official text.

| Field | Official value |
|---|---|
| Problem statement ID | 26265 |
| Title | Streamline of security deposit/ FDR portal |
| Organisation / department | AICTE, Regulation Bureau (Ministry of Education) |
| Category / theme | Software / Smart Automation |
| Users named | Regulation Bureau officials, other authorised stakeholders, institutions |

**Background in one line.** Institutions may have to submit or maintain security deposits or FDRs for AICTE approval. Records sit in fragmented files, correspondence and several trackers, so current status, pending actions and deadlines are hard to see.

**Where the official text differs from the research framing.**

- It never says "fraud", "10-year lock-in" or "lien". Those came from our own prompt and the AI reports. Keep them as design drivers, but do not present them to judges as the stated requirement.
- Bank checks are to be used "wherever supported" and integration "where technically feasible and formally authorized". A clearly labelled simulated bank check is therefore a defensible prototype choice (our reading).
- Release is "recording and tracking of requests", subject to authorisation by the competent authority. The portal must record and route the decision. It does not need to move money.

- Success is measured, not described: record accuracy, verification turnaround, timely renewal monitoring, less manual tracking effort, and visibility into pending cases.

**Six test scenarios the statement names.** New FDR submission, validity tracking, approaching maturity, renewal, discrepancy identification, and authorised release processing. The demo and the test suite should be built around exactly these.

## 2. Requirement breakdown

The statement lists ten expected capabilities. Each becomes one requirement below with an acceptance test, so "done" is checkable. Priority is our judgment: Must items are needed for the six test scenarios.

| ID | Requirement | Acceptance test | Priority |
|---|---|---|---|
| R1 | Central repository, institution-wise and instrument-wise | Every record holds reference number, bank, amount, issue date, maturity date, validity status, documents and administrative status. Search by institution, bank, status or date range returns correct rows. | Must |
| R2 | Digital document management | Upload, versioned storage and retrieval of FDR copies and deposit confirmations. Each file has a stored SHA-256 hash. Access follows role. | Must |
| R3 | Verification and discrepancy detection | Each key field shows two values: submitted by the institution and confirmed by the bank. Mismatches and missing items are flagged with a reason code. | Must |
| R4 | Validity and maturity tracking | Status is computed from dates. Reminders fire at configurable thresholds before maturity or expiry. | Must |
| R5 | Renewal and replacement monitoring | A renewed or replacement instrument links to its predecessor. The full history stays visible. | Must |

| ID | Requirement | Acceptance test | Priority |
|---|---|---|---|
| R6 | Release and refund workflow | Request, scrutiny, competent-authority approval and bank-side confirmation are separate steps. Every step is logged. A blocked case (dispute, pending complaint) cannot be approved. | Must |
| R7 | Alerts and escalation | Pending verification, approaching deadline, overdue and unresolved cases notify the right people. Unacknowledged alerts escalate on a schedule. | Must |
| R8 | Dashboard and reports | Counts for active, expiring, matured, renewed, released and pending instruments. Reports for pending verifications, upcoming maturities and missing documents. | Must |
| R9 | Audit trail and access control | Role-based access. Every submission, verification, change and approval is recorded with actor and time. Edits to history are detectable. | Must |
| R10 | Integration and data security | Bank verification and AICTE systems connect through replaceable adapters. Data is encrypted in transit and at rest. | Should |

**Implied requirements the statement states in passing.**

- Officials can record scrutiny observations on a case.
- Permitted fields can be edited, and every edit is kept in history.
- Reminder timing is configurable, not fixed in code.
- Institutions see their own status and pending actions (needed for the "timely alerts to institutions" goal).

## 3. Quality analysis of the three reports

Use the Gemini report for AICTE-specific rules, the Claude report for what is actually verified, and the ChatGPT report only as a short sanity check. No report alone is safe to build from: they disagree on five points that decide the architecture, and the one with the most detail makes the boldest unverified claims.

*Note on the files:* the "Claude" file you attached (.md) is a corrupted PDF and could not be read. I reviewed the original text of the same report from this session instead. The ratings below are my judgment, 5 = best.

| | Claude report | Gemini report | ChatGPT report |
|---|---|---|---|
| Size and shape | About 4,200 words, five sections, inline source links | 25 pages, 76 works cited, diagrams, cron table, state diagram | 3 pages, bullets, 4 references |
| Source honesty | 5: marks each source as read or search-only, labels inference | 2: many cites are secondary or unrelated | 3: cites a few real papers, leans on Wikipedia for definitions |
| AICTE specificity | 3: found tenure and penalty clauses but not the full tables | 5: Formats 3, 4 and 6, Affidavit 9, bank-merger NOC, dispute hold, Women's-institution rule | 2: generic |
| Technical depth | 4: scheduler idempotency, reconciliation classes, audit hashing | 4: cron jobs, state machine, refund formula | 2: one paragraph per layer |
| Risk of building on a wrong claim | Low | High | Medium |
| Best use for us | Architecture decisions, verification limits, legal weight of audit logs | Business rules and lifecycle detail, after checking them against the handbook | Minimal state list, quick stack options |

### What each report does well

- **Claude:** shows the bank, not an open-banking feed, must be the authority on lien status, because the Account Aggregator term-deposit schema has no lien field and AICTE cannot be a Financial Information User. Adds documented fraud cases and a renewal-lineage design.
- **Gemini:** the fullest lifecycle (origination, verification, custody, default, release), the DISPUTE_HOLD and PENALTY_ASSESSED states, a two-step discharge that waits for the bank's confirmation (mirroring NeSL), and a piecewise refund rule.
- **ChatGPT:** a clean four-step release flow (Submitted, Bank-verified, Pending approval, Released) and correct stack basics. Its four references (Treleaven and Batrinca 2017, Becker et al. 2020, Larsen and Gilani 2017, van Liebergen 2017) are real RegTech papers as far as I recall; I did not re-verify them here.

### Where each report is weak

- **Claude:** could not retrieve the SIH text or AICTE's deposit amounts, found no explicit "lien" wording in any handbook, and has no peer-reviewed paper on FDR lien APIs.
- **Gemini:**
  - Its headline numbers (12% manual error, 0.17 seconds per document, 98.5% accuracy, 40% faster) come from a study of identity documents (Aadhaar, PAN, voter ID, 1,500 samples), not bank FDRs. Quote them only as indicative.
  - It presents NeSL's Notification, Document Pull and Event Request APIs as usable for FDRs. Those are built for electronic bank guarantees.
  - It describes lien-management and lien-release APIs with signed receipts, and bank webhooks, without a verified source. The Claude report found FD create, fetch and liquidate APIs but no lien API.
  - The penalty rule (10% after an Extension of Approval lapse, 50% after six years) cites a Scribd copy of the handbook and differs from the clause the Claude report cites (10% for premature encashment).
  - The works-cited list includes unrelated items, such as a college balance sheet, a court cause list, a tender notice and a company prospectus. A long list is not evidence.
  - Permissioned blockchain, behavioural authentication and deep-learning document forensics are heavy for a prototype.
- **ChatGPT:** none of its references covers liens, bank APIs or state machines, so parts (b) and (c) of the reference ask are unanswered. It suggests Account Aggregator for FDR checks without noting the schema limit, and mixes in foreign examples (Plaid, Yodlee,

Escrow.com).

### Where they disagree, and the call we should make

| Topic | Claude | Gemini | ChatGPT | Our decision |
|---|---|---|---|---|
| How to verify with the bank | No public lien-verification API. Use a bank maker-checker role and a daily bank file | Open-banking and NeSL-style lien APIs | Account Aggregator or NeSL-style pull | One bank-adapter interface. Ship a simulated bank (maker-checker plus file feed) now. Real API later |
| Deposit tenure | Varies by handbook edition (8 or 10 years) | Fixed 10 years | Fixed 10 years | Tenure stored per record |
| Penalty rules | Premature encashment 10%; late proof 10% or 50% | EoA lapse 10%, six years 50% | Not covered | Penalty rules live in configuration. Check them against the handbook before the demo |
| Document forensics | Signature and QR first, OCR advisory only | YOLO and CNN-OCR as the core check | Not covered | OCR only to pre-fill fields. Trust comes from the bank confirmation |
| Audit log | Hash chain, head hash anchored outside the database | Permissioned blockchain | Append-only log or blockchain | Hash chain. Blockchain adds nothing for one authority |
| Scheduler | One idempotent daily job, alert stage computed from dates | Four fixed cron jobs | Plain cron | Idempotent daily job plus a monthly bank-merger check |

# 4. Existing solutions and how they work

No live system lets a regulator verify a third party's FDR and its lien through an open API. The closest working patterns each cover one slice, and NeSL appears in all three reports. Links come from the Claude report, whose notes say these pages were opened.

| Solution | How it works | How it reaches the bank | What we borrow | Limit |
|---|---|---|---|---|
| NeSL e-Bank Guarantee (live since 2022, 38 banks) | Bank issues, e-stamps and e-signs a guarantee on NeSL's platform. The beneficiary gets it instantly and can raise invocation, amendment or cancellation. | Beneficiary systems use three APIs: Notification, Document Pull, Event Request. Bank-side connection method is not public. | Neutral record, bank as origin, beneficiary pull and notify. A status closes only when the bank confirms. | It is a guarantee, not a lien on a deposit. |
| Clearing-corporation e-FDR (Bank of Baroda, NSE Clearing) | Bank marks the lien instantly for the clearing corporation, blocks printing and manual lien removal. The clearing corporation releases the lien by API. | Closed bilateral link with empanelled banks. | The right model for an AICTE-to-bank link: lien set by the bank, release only on the holder's instruction. | No public API for third-party verification. Empanelment needed. |

| Solution | How it works | How it reaches the bank | What we borrow | Limit |
|---|---|---|---|---|
| Account Aggregator | Consent-based sharing of term-deposit data: maturity date and amount, principal, rate, tenure, masked account, holder type. | Regulated aggregators route encrypted data to approved information users. | Field names for amount and maturity checks. Consent and signing pattern. | No lien or status field. AICTE cannot be an information user under the RBI rules. |
| DigiLocker issuer workflow | Issuers push or pull digitally signed documents. Users retrieve their own. | Issuer integration with NeGD. | Signed-document trust model, legal footing for e-documents. | No evidence that any bank issues FDRs this way. Users fetch only their own documents. |
| Today's AICTE process | Paper or PDF copy, affidavit, Excel and file registers. | Letters and email to branches, ad hoc. | None. This is the baseline to beat. | Nothing proves a file came from the bank. |

**Unconfirmed claims to keep out of the pitch.** The Gemini report adds SFMS (the banks' messaging network) as an FDR-confirmation rail, and the Claude report found only a fee-schedule hint of it. Treat SFMS as a production-stage question for a bank partner, not a prototype feature.

**Why NeSL is still the best analogue.** Its flow is the one we want to copy at small scale: the bank creates the record, the beneficiary is notified, and the beneficiary asks for a change through an event that the bank must act on. The Department of Expenditure's August 2022 amendment to the General Financial Rules (rules 170(i) and 171(i)) shows central government already accepts electronic security instruments, though it does not mandate NeSL.

# 5. Gaps

The gap is the regulator's side of the relationship: a record that tracks an instrument for years, shows who vouched for each fact, and routes release through several offices. Existing platforms verify issuance for the bank and beneficiary; none gives AICTE that view.

| # | Gap | Evidence | What it means for the design |
|---|---|---|---|
| G1 | A file carries no proof it came from the bank | Documented FDR frauds were found at encashment, not at submission, including an AICTE case where a fake Rs 35 lakh FDR dated 2007 led to a CBI referral in 2012 (Lok Sabha reply) | Every key field needs a source label: submitted or bank-confirmed |
| G2 | No public rail verifies a third party's FDR and lien | Account Aggregator schema has no lien field; AICTE cannot be an information user | Bank confirmation must be a pluggable adapter. The prototype simulates it |
| G3 | Renewal changes the FDR number and date | Bank lien letters must cite old and new numbers after renewal (MSEI template) | Model each instrument as a chain of predecessor and successor, not one row per FDR number |
| G4 | Premature closure at the bank goes unnoticed | Handbook penalises early encashment, but a lien letter does not alert AICTE | Daily comparison against the bank feed, with an incident alert on mismatch |
| G5 | Excel and regional files give no single source of truth | All three reports describe fragmented tracking; Excel rows go stale after renewal | One database, one history, one dashboard |
| G6 | Ten-year horizon outlasts staff tenure | A custodian-dependence case in Chandigarh (Tribune) | Alerts go to roles and queues, never to one person |

| # | Gap | Evidence | What it means for the design |
|---|---|---|---|
| G7 | Release needs sequential sign-offs across offices | Gemini lifecycle: bureau, grievance cell, regional office, finance | Parallel clearance checklist, with blocking flags (dispute, pending complaint) |
| G8 | Mixed population: 8- and 10-year tenures, online and bank-issued deposits | Claude report on handbook editions | Per-record tenure and instrument type |
| G9 | No public data on AICTE's own delays or error rates | Claude report: none found | We must measure our own baseline in the demo, not quote someone else's figure |

G1 to G8 come from documented cases and rule text. The AICTE-specific mechanisms are inferences, since no published audit of AICTE's tracking exists.

## 6. What the system wants from us, and our USP

Judges will look for a working lifecycle, visible trust, and measured improvement, not for the most technology. We win by making three things obvious on screen: who vouched for each fact, nothing missed on a date, and nothing released without a trail.

**What the system (the statement and its evaluators) wants.**

1. A working end-to-end path through the six named scenarios, on realistic data.
2. Visible separation of institution-submitted data from independently confirmed data.
3. Reminders and overdue flags that fire on time and can be configured.
4. A release or refund request that moves through authorised steps and cannot skip one.
5. A dashboard with the six status counts and the reports named in the statement.
6. Role-based access, encryption and an audit trail that stands up to questions.
7. Evidence of improvement: record accuracy, verification turnaround, renewal timeliness, manual effort, visibility.

**Our USPs.** Each one answers a gap above and can be shown in under a minute.

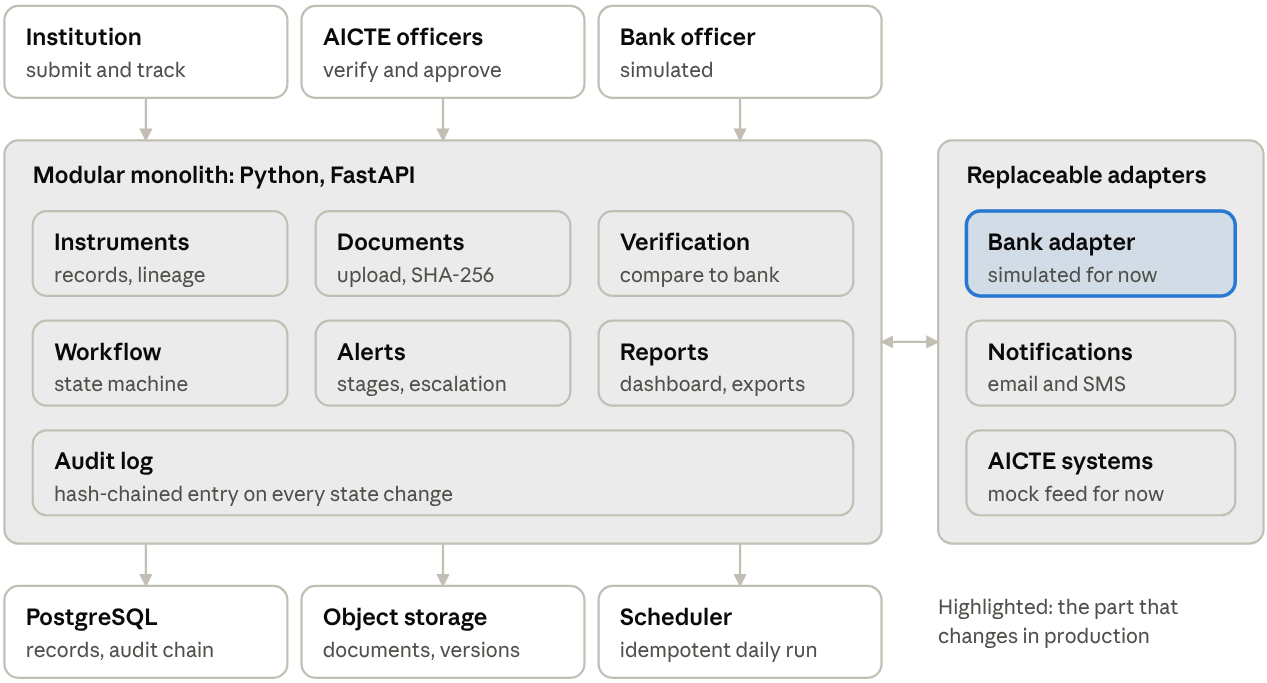
| USP | What judges see | Gap it closes | Effort |
|---|---|---|---|
| Two-column trust view: Submitted vs Bank-confirmed on every field, with a trust badge per instrument | A case where amount matches but maturity differs lights up red with a reason code | G1, G2 | Medium |
| Pluggable bank adapter with a simulated bank console (maker-checker) and a daily file feed | Bank officer confirms in one window, AICTE officer sees the status change in another | G2, G4 | Medium |
| Renewal lineage | Old and new FDR numbers shown as one chain with all documents | G3 | Small |
| Stage-based alerts with an escalation ladder and a demo clock | Move the clock 30 days forward and watch alerts and escalations fire once each, never twice | G4, G6 | Medium |
| Release workflow with blocking guards | Release is blocked while a dispute flag is open, and the screen says why | G7 | Medium |
| Tamper-evident audit log with one-click evidence pack | Edit a database row by hand and the integrity check fails; export a signed pack for a case | G5 | Medium |
| Impact panel: baseline vs portal | Measured minutes per verification and per release request, from a scripted run | G9 | Small |
| Rules in configuration | Change a reminder threshold or penalty percentage without code | G8 | Small |

**What we deliberately skip in the prototype.** Blockchain, deep-learning forgery detection, and live bank or Account Aggregator calls. Each adds risk without answering the statement, and the architecture leaves a clean place to add them.

# 7. Target architecture and data model

Build one modular monolith on PostgreSQL, with a replaceable bank adapter and a state machine that only a bank confirmation can advance. This is a design proposal assembled from the three reports; section 3 explains why each choice beat its alternative.

**One monolith and one database, with a bank adapter we can swap later**




architecture · 3 user types, 6 modules, 3 adapters

Officers and the simulated bank officer work through the same portal. Only the bank adapter has to change to reach a real bank.

## Stack decisions

| Layer | Choice | Why | Alternative |
|---|---|---|---|
| Frontend | React with TypeScript; accessibility tests in CI | Large hiring pool for a student team. GIGW 3.0 (built on WCAG 2.1) is the baseline for government sites | Angular, also seen in NeGD job ads |

| Layer | Choice | Why | Alternative |
|---|---|---|---|
| Backend | Python with FastAPI, one modular monolith | Quick to build; PDF-signature and OCR libraries are strong in Python; avoids microservice overhead | Java with Spring Boot, the stack in NeGD job ads, if the team prefers it |
| Database | PostgreSQL | Relational model of institutions, instruments and lineage; ACID transitions | None needed |
| Documents | S3-compatible storage (MinIO for the prototype), SHA-256 hash per version | Versioned files, hash recorded in the database | Cloud bucket in an Indian MeitY-empanelled region |
| Scheduler | One idempotent daily job (APScheduler or Celery beat) | A missed run catches up; the unique key stops duplicate alerts | Temporal or Camunda if long-running workflows grow; Kubernetes CronJob |
| Workflow | Pure function (state, event) gives (new state, effects), with a transition table in code | Testable; can move into a workflow engine later | Camunda if staff must read the diagram |
| Audit | Append-only table; each row hashes the previous row plus its own JSON; head hash exported daily | Tamper-evident and cheap; supports an electronic-evidence certificate (Claude report cites section 63(4) of the Bharatiya Sakshya Adhiniyam 2023) | Permissioned blockchain, not needed for one authority |
| Identity | Keycloak (OIDC), MFA for officers, role plus institution-scoped access | Standard, replaceable by government SSO later | Existing AICTE login |

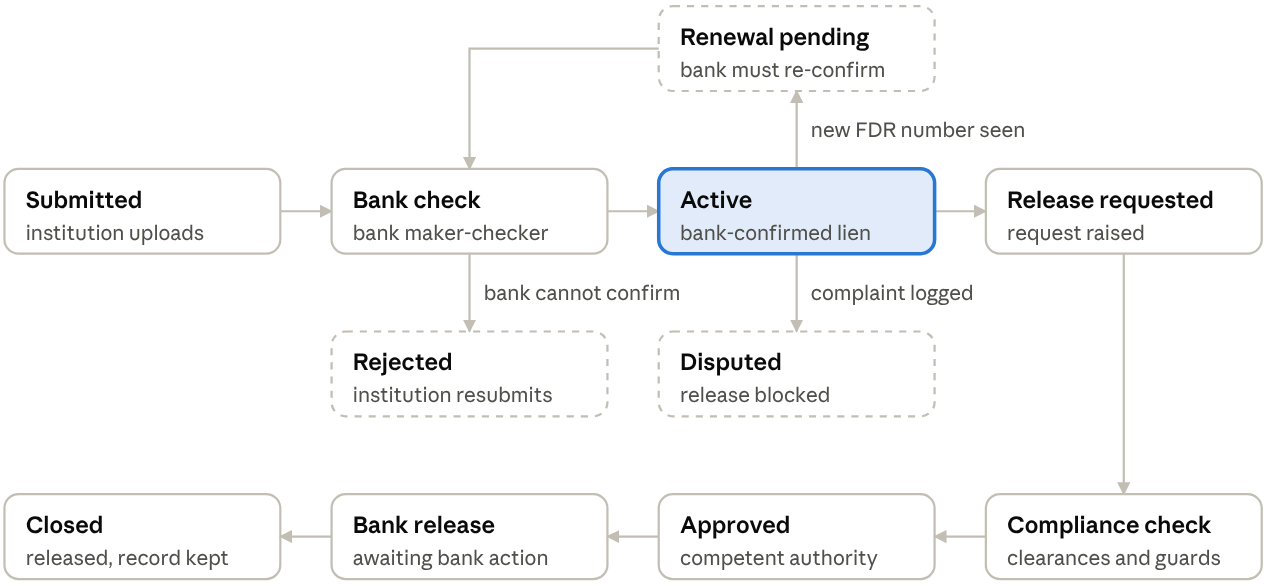
| Layer | Choice | Why | Alternative |
|---|---|---|---|
| Security | TLS everywhere, AES-256 for sensitive fields, India-hosted data, ZAP, Semgrep and Trivy scans in CI | Matches the CERT-In and DPDP points in the Claude report | None |

### Core data model

| Entity | Key fields | Note |
|---|---|---|
| Institution | id, name, promoter body, contacts | Owns many instruments |
| Instrument | id, institution, type (bank-issued FDR or online deposit), bank, reference number, amount in paise, issue date, maturity date, tenure in years, status, predecessor id | Predecessor id builds the renewal chain |
| Field confirmation | instrument, field, submitted value, confirmed value, source, confirmed by and at, result | The two-column trust view reads from here |
| Document | id, instrument, version, SHA-256, uploaded by and at | Never overwritten |
| Alert | instrument, stage, channel, due at, sent at, acknowledged at | Unique on (instrument, stage, channel) |
| Release request | instrument, kind (release, refund, closure), checklist, decision, decided by, bank confirmed at | Approval needs a maker and a different checker |
| Audit entry | sequence, actor, role, entity, from state, to state, payload hash, previous hash, row hash, time | Written in the same transaction as the change |
| Rule | key, value, clause reference | Thresholds, escalation days, penalty percentages |

**State machine.** Each instrument moves through the states below. Every arrow has an actor role and a guard, and the transition and its audit entry commit together.

**Nothing becomes Active without a bank check, and nothing closes without one**




Dashed boxes are exception paths. The highlighted state is the only one that lets a release start.

instrument states · 8 normal, 3 exception

The normal path is solid. The three dashed boxes send a case back to the bank or to a person. Disputed returns to Active when the dispute is cleared.

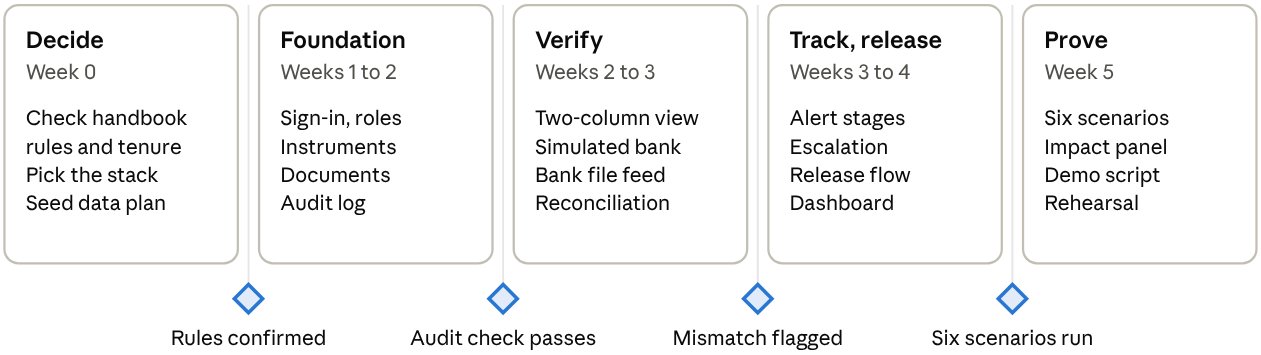
**Rules the scheduler and reconciler follow (starting values, all configurable).**

- Alert stages: 180, 90, 60, 30 and 7 days before maturity, on the day, and overdue. Stage is a label on the instrument, not a state. The job picks the largest threshold already crossed, so a missed day still alerts the next day.
- Escalation ladder for unacknowledged alerts: institution after 1 day, officer after 7, senior or finance after 15, head and auditor after 30.
- Bank comparison outcomes: MATCH, AMOUNT_MISMATCH, MATURITY_MISMATCH, LIEN_MISSING, CLOSED_AT_BANK, NOT_FOUND. A renewal seen in the bank file never auto-accepts; it sends the instrument to Renewal pending.
- Release guards (placeholders until the handbook is checked): no open dispute, no pending complaint, required clearances complete, maker and checker are different people, approval is signed.

# 8. Implementation plan

Build in five phases, each ending in a gate that can be shown on screen, so a late phase never rests on an unproven one. Week numbers are an assumption because the hackathon dates are not known; compress or stretch them to fit.

**Five phases, four gates: nothing moves on until the one before it works**




roadmap · 5 phases, 4 gates, weeks assumed

A diamond is a gate: the criterion under it must be demonstrated before the next phase starts.

### What to build in each phase

| Phase | Build | Done when |
|---|---|---|
| 1. Decide | Confirm handbook rules and tenure, choose the stack, write the seed-data spec | The open questions in section 9 are answered, or the assumption is written down |
| 2. Foundation | Sign-in with roles, institutions and instruments, document upload with hash, audit log with hash chain, CI with tests | Hand-editing a database row makes the integrity check fail |
| 3. Verify | Field confirmation model, submitted vs confirmed view, simulated bank console, bank file import, comparison outcomes | A seeded mismatch shows red with a reason code, from upload to flag |

| Phase | Build | Done when |
|---|---|---|
| 4. Track and release | Alert stages, escalation, demo clock, renewal lineage, release workflow with guards, dashboard and reports | Moving the demo clock fires each alert once; a dispute blocks release |
| 5. Prove | 20 to 50 seeded institutions, the six scenario scripts, impact panel, accessibility and security scans, rehearsal | The full demo runs twice in a row with no manual fixes |

### Demo script: the six scenarios from the statement

| # | Scenario | What happens on screen | Evidence shown |
|---|---|---|---|
| 1 | New FDR submission | Institution uploads an FDR copy and enters its details | Record in Submitted state, document hash stored, audit entry written |
| 2 | Discrepancy identification | Bank officer confirms a different maturity date | Field turns red with MATURITY_MISMATCH and the case is flagged for scrutiny |
| 3 | Validity tracking | Bank confirms the correct values | Instrument becomes Active; dashboard counts update |
| 4 | Approaching maturity | Demo clock moves forward | Stage alerts fire once each; unacknowledged ones escalate |
| 5 | Renewal | Bank file shows a new FDR number | Instrument goes to Renewal pending, new record links to the old one, bank re-confirms |
| 6 | Authorised release | Release requested, dispute flag blocks it, flag cleared, maker and checker approve, bank confirms | Instrument Closed; full trail and a signed evidence pack exported |

**How we prove "measurable improvement".** The statement asks for it, and no public data on AICTE's own delays exists, so we measure our own before-and-after on synthetic data and say so.

| Measure from the statement | How we measure it | Baseline |
|---|---|---|
| Record accuracy | Inject a known number of errors into the seeded records; report how many the system flags | Teammate checks the same records by eye in a spreadsheet |
| Verification turnaround | Minutes from submission to bank-confirmed status | Timed manual walkthrough with PDF and email |
| Timely renewal monitoring | Share of seeded maturities that raised an alert before the due date | Spreadsheet with no alerts: count how many a volunteer catches |
| Manual tracking effort | Time and clicks to answer "which FDRs mature in 90 days?" | Filtering the Excel sheet |
| Visibility of pending cases | Time to list every pending verification | Same question on the spreadsheet |

Report the numbers we measure and nothing else, and label them as results on synthetic data.

## 9. Risks, assumptions and open questions

The biggest risk is promising a bank integration we cannot show. Every item below has a fallback that keeps the demo honest.

| Risk | Why it matters | Mitigation |
|---|---|---|
| Judges expect live bank verification | The statement says "wherever supported", but a demo with fake banks can look hollow | Label the simulated bank clearly, show the adapter interface and the file-feed format, and explain the production path (nodal bank API or daily file) |

| Risk | Why it matters | Mitigation |
|---|---|---|
| Penalty and release rules are wrong | Reports disagree on clauses and percentages | Keep them in configuration, check against the official handbook before the demo, and show the clause reference on screen |
| Scope creep into AI forensics or blockchain | Eats build time, adds unverifiable claims | Out of scope for the prototype; list as future work |
| Scheduler fires twice or misses a day | Undermines the main selling point | Idempotency key on (instrument, stage, channel); catch-up rule uses the largest threshold crossed; test with the demo clock |
| Personal and financial data handling | DPDP Act obligations start 13 May 2027, per the Claude report | Minimise and mask personal data, encrypt at rest, use synthetic data in the demo |
| Fabricated or unsourced statistics in our pitch | Judges may check | Quote only figures we measured, or sources we opened; mark the identity-document study as indicative |

**Open questions to settle before building.**

- [ ] Which handbook edition applies to the demo data (8-year or 10-year tenure), and what are the official deposit amounts per programme?
- [ ] What are AICTE's actual release conditions and the order of clearances? The guards in section 7 are placeholders until confirmed.
- [ ] Does AICTE's web portal expose any data we can read, or do we mock it?
- [ ] Team language and framework: the stack in section 7 assumes Python and React. Say if the team prefers Java.
- [ ] Hackathon dates and demo length, so the plan in section 8 can be pinned to real dates.
- [ ] Is a single institution view enough for the demo, or must it show many institutions at once (we assume 20 to 50 seeded)?